1. Why Conditions Matter in Data Engineering

After reading and cleaning data, a pipeline needs to make decisions.

For example:

- Should a record be loaded or rejected?
- Is a required field missing?
- Is the order amount valid?
- Is the status allowed?
- Should the pipeline be marked `SUCCESS`, `PARTIAL_SUCCESS`, or `FAILED`?

Conditions help convert these business rules into Python logic.

---

2. Basic `if` Condition

Use `if` when some code should run only when a condition is true.

```
order_amount = 2500

if order_amount >= 0:
    print("Valid order amount")
```

Output:

```
Valid order amount
```

If the condition is false, the block is skipped.

---

3. `if` and `else`

Use `else` when you need two possible outcomes.

```
order_amount = -500
```

```
if order_amount >= 0:
    print("Valid order")
else:
    print("Invalid order")
```

Output:

Invalid order

Think of it as:

Condition True → IF block
Condition False → ELSE block

---

4. Real Data Quality Rule

Suppose `customer_id` is mandatory.

```
customer_id = None

if customer_id is None:
    print("Reject Record")
else:
    print("Customer ID Valid")
```

Output:

Reject Record

This is a direct example of converting a business rule into code.

Business Rule

Customer ID cannot be missing.

Python Rule

```
if customer_id is None:
    record_status = "REJECTED"
```

---

5. Using `elif`

Use `elif` when more than two outcomes are possible.

Suppose the business rule is:

0% rejected → SUCCESS
Up to 5% rejected → PARTIAL_SUCCESS
More than 5% → FAILED

Code:

```
rejection_percentage = 3.5

if rejection_percentage == 0:
    pipeline_status = "SUCCESS"
elif rejection_percentage <= 5:
    pipeline_status = "PARTIAL_SUCCESS"
else:
    pipeline_status = "FAILED"
```

Result:

PARTIAL_SUCCESS

---

6. Condition Order Is Important

Python checks conditions from top to bottom.

Consider:

```
rejection_percentage = 0
```

```
if rejection_percentage <= 5:
    pipeline_status = "PARTIAL_SUCCESS"
elif rejection_percentage == 0:
    pipeline_status = "SUCCESS"
```

The result becomes:

PARTIAL_SUCCESS

Why?

Because:

0 <= 5

is already `True`.

Python never reaches the next condition.

Correct Order

```
if rejection_percentage == 0:
    pipeline_status = "SUCCESS"
elif rejection_percentage <= 5:
    pipeline_status = "PARTIAL_SUCCESS"
else:
    pipeline_status = "FAILED"
```

Key Rule

Place more specific conditions before broader conditions.

---

7. Combining Multiple Conditions

A record may be considered valid only when multiple rules pass.

Example rules:

- Customer ID must exist

- Order amount cannot be negative
- Status must be valid

```
customer_id = 101
order_amount = 2500
status = "ACTIVE"

if (
    customer_id is not None
    and order_amount >= 0
    and status in ["ACTIVE", "INACTIVE"]
):
    print("VALID RECORD")
else:
    print("REJECTED RECORD")
```

Output:

VALID RECORD

This pattern is very common in Data Engineering validations.

---

8. Record-Level Validation

Suppose a customer record must follow these rules:

1. Customer ID cannot be missing.
2. Customer name cannot be empty.
3. Order amount cannot be negative.
4. Status must be `ACTIVE` or `INACTIVE`.

```
customer_id = 101
customer_name = "Anurag"
order_amount = 2500
status = "ACTIVE"
```

Validation:

```
if customer_id is None:
```

```
    record_status = "REJECTED"
    rejection_reason = "Customer ID missing"

elif customer_name.strip() == "":
    record_status = "REJECTED"
    rejection_reason = "Customer name missing"

elif order_amount < 0:
    record_status = "REJECTED"
    rejection_reason = "Invalid order amount"

elif status not in ["ACTIVE", "INACTIVE"]:
    record_status = "REJECTED"
    rejection_reason = "Invalid customer status"

else:
    record_status = "VALID"
    rejection_reason = None
```

Expected result:

Record Status: VALID
Rejection Reason: None

---

9. Rejection Reason Is Important

A rejected record should not only be marked `REJECTED`.

It is useful to also capture **why** it was rejected.

Example:

order_amount = -500

Result:

REJECTED
Invalid order amount

Another example:

status = "UNKNOWN"

Result:

REJECTED
Invalid customer status

This makes debugging and rejected-record analysis easier.

---

10. Nested Conditions

A condition can exist inside another condition.

```
file_received = True
schema_valid = False

if file_received:
    if schema_valid:
        print("Start Processing")
    else:
        print("Schema Validation Failed")
else:
    print("Source File Missing")
```

Output:

Schema Validation Failed

This is called a **nested condition**.

However, avoid unnecessary nesting.

Often this is cleaner:

```
if file_received and schema_valid:
    print("Start Processing")
else:
```

```
    print("Pipeline Validation Failed")
```

Readable code is easier to maintain.

---

11. Truthy and Falsy Values

Python can evaluate values directly in conditions.

Common falsy values include:

```
None
False
0
""
[]
{}
```

Example:

```
customer_name = ""

if customer_name:
    print("Customer name available")
else:
    print("Customer name missing")
```

Output:

```
Customer name missing
```

Another example:

```
error_message = None

if error_message:
    print("Error Found")
else:
    print("No Error")
```

Output:

No Error

---

12. Be Careful with Truthy/Falsy Checks

Consider:

retry_count = 0

This:

```
if retry_count:
    print("Retry information available")
else:
    print("Retry information missing")
```

treats `0` as false.

But `0` may be a perfectly valid retry count.

If the requirement is specifically to check whether the value is missing, use:

```
if retry_count is None:
    print("Retry count missing")
else:
    print("Retry count available")
```

Important

0 does not automatically mean missing.
False does not automatically mean missing.

Use explicit checks when business meaning matters.

---

13. Pipeline-Level Business Rules

Conditions are also used to decide the status of an entire pipeline.

Suppose:

source_count = 1500
target_count = 1475

file_received = True
schema_valid = True
error_message = None

Calculate rejected records:

rejected_count = source_count - target_count

Calculate rejection percentage safely:

```
if source_count > 0:
    rejection_percentage = (
        rejected_count / source_count
    ) * 100
else:
    rejection_percentage = 0
```

---

14. Pipeline Status Logic

Suppose the rules are:

- Missing source file → `FAILED`
- Invalid schema → `FAILED`
- Existing error → `FAILED`
- More than 5% rejected → `FAILED`
- Some records rejected but within limit → `PARTIAL_SUCCESS`
- No rejected records → `SUCCESS`

Implementation:

```
if not file_received:
    pipeline_status = "FAILED"
    reason = "Source file missing"

elif not schema_valid:
    pipeline_status = "FAILED"
    reason = "Schema validation failed"

elif error_message is not None:
    pipeline_status = "FAILED"
    reason = error_message

elif rejection_percentage > 5:
    pipeline_status = "FAILED"
    reason = "Rejection threshold exceeded"

elif rejection_percentage > 0:
    pipeline_status = "PARTIAL_SUCCESS"
    reason = "Some records rejected"

else:
    pipeline_status = "SUCCESS"
    reason = None
```

For:

Source Count: 1500
Target Count: 1475
Rejected Count: 25
Rejection Percentage: 1.67%

Result:

Pipeline Status: PARTIAL_SUCCESS
Reason: Some records rejected

---

15. Business Requirement → Python Condition

This is the most important concept from this class.

Business Requirement

Reject records where customer ID is missing.

```
if customer_id is None:
    record_status = "REJECTED"
```

Business Requirement

Allow maximum 5% rejected records.

```
if rejection_percentage > 5:
    pipeline_status = "FAILED"
```

Business Requirement

Only CSV and JSON files are supported.

```
if file_name.endswith((".csv", ".json")):
    print("Supported file")
```

Key Idea

A Data Engineer should be able to take a written business rule and convert it into readable validation logic.

---

16. Record Validation vs Pipeline Validation

Record-Level Validation

Checks whether an individual record is valid.

Examples:

- Missing customer ID
- Empty customer name
- Negative order amount
- Invalid customer status

Pipeline-Level Validation

Checks whether the overall pipeline should continue or fail.

Examples:

- Source file missing
- Schema invalid
- Error exists
- Rejection threshold exceeded

Both use the same `if`, `elif`, and `else` concepts.

---

17. Common Mistakes

Mistake 1: Multiple `if` Statements for Mutually Exclusive Outcomes

Avoid:

```
if percentage == 0:
    status = "SUCCESS"

if percentage <= 5:
    status = "PARTIAL_SUCCESS"
```

The second condition can overwrite the first result.

Prefer:

```
if percentage == 0:
    status = "SUCCESS"
elif percentage <= 5:
    status = "PARTIAL_SUCCESS"
```

```
else:
    status = "FAILED"
```

Mistake 2: Wrong Condition Order

Avoid placing broad conditions before specific ones.

Wrong:

```
if percentage <= 5:
    status = "PARTIAL_SUCCESS"
elif percentage == 0:
    status = "SUCCESS"
```

Put the exact condition first.

Mistake 3: Treating `0` as Missing

Avoid:

```
if not retry_count:
    print("Missing")
```

when zero is a valid value.

Prefer:

```
if retry_count is None:
    print("Missing")
```

Mistake 4: Too Much Nesting

Deep nested conditions make pipeline code difficult to read.

Prefer flatter and clearer conditions when possible.

---

18. Quick Revision

- `if` checks a condition.
- `else` handles the alternative case.
- `elif` handles additional conditions.
- Python checks `if/elif` conditions from top to bottom.
- The first matching condition is executed.
- Specific conditions should usually come before broader ones.
- Multiple rules can be combined with logical operators.
- Record validations can produce rejection reasons.
- Truthy/falsy checks should be used carefully.
- `0` and `False` may be valid values.
- Use `is None` when specifically checking missing values.
- Conditions convert business rules into executable pipeline logic.

---

19. Student Practice

Given:

order_id = 5001
customer_id = 101
order_amount = -200
payment_status = "PAID"

Business rules:

- `order_id` cannot be missing.
- `customer_id` cannot be missing.
- `order_amount` cannot be negative.
- `payment_status` must be one of:

PAID
PENDING
FAILED

Create validation logic that produces:

VALID

or:

REJECTED

along with a rejection reason.

For the given record, expected result:

REJECTED
Invalid order amount

---

20. Additional Practice Scenario

Try:

order_id = 5002
customer_id = None
order_amount = 1500
payment_status = "PENDING"

Think about:

1. Which rule fails first?
2. What rejection reason should be stored?
3. Should later conditions still be checked after one matching `elif`?

---

21. Interview-Relevant Questions

1. What is the difference between `if`, `elif`, and `else`?

`if` checks the first condition.

`elif` checks additional conditions when previous conditions fail.

`else` runs when none of the earlier conditions are true.

---

2. Why does condition order matter?

Python evaluates conditions from top to bottom and stops after the first matching branch.

Because of this, more specific business rules should usually be placed before broader rules.

---

3. What are truthy and falsy values?

Some values are automatically interpreted as false in Boolean conditions.

Examples include:

None
False
0
""
[]
{}

---

4. Why can `if not value` be dangerous in data validation?

Because valid business values such as `0` and `False` are also falsy.

If the requirement is to check missing data specifically, use an explicit condition such as:

value is None

---

5. How are conditional statements used in Data Engineering?

They implement:

- Mandatory-field validation
- Data-quality checks
- Rejection thresholds
- Schema checks
- File validation
- Pipeline success/failure logic

---

6. Why should rejection reasons be captured?

They help explain why a record failed validation and make debugging and rejected-record analysis easier.

---

7. When should `elif` be preferred over multiple `if` statements?

Use `elif` when only one outcome should be selected from multiple mutually exclusive conditions.

---

8. What is a nested condition?

A nested condition is an `if` statement inside another `if` statement.

It can be useful when one validation depends on another, but unnecessary nesting should be avoided for readability.

---

22. Mini Interview Challenge

Predict the final status:

```
rejection_percentage = 0

if rejection_percentage <= 5:
    status = "PARTIAL_SUCCESS"
elif rejection_percentage == 0:
    status = "SUCCESS"
else:
    status = "FAILED"

print(status)
```

Answer:

PARTIAL_SUCCESS

Why?

Because `0 <= 5` is already `True`, so Python does not evaluate the `elif`.

The correct order should be:

```
if rejection_percentage == 0:
    status = "SUCCESS"
elif rejection_percentage <= 5:
    status = "PARTIAL_SUCCESS"
else:
    status = "FAILED"
```

---

23. Key Takeaway

Conditions allow Data Engineers to convert business requirements and data-quality rules into executable pipeline logic.

Do not learn `if`, `elif`, and `else` only as Python syntax.

Think in terms of:

Business Rule

↓
Validation Condition
↓
Decision
↓
VALID / REJECTED
or
SUCCESS / PARTIAL_SUCCESS / FAILED